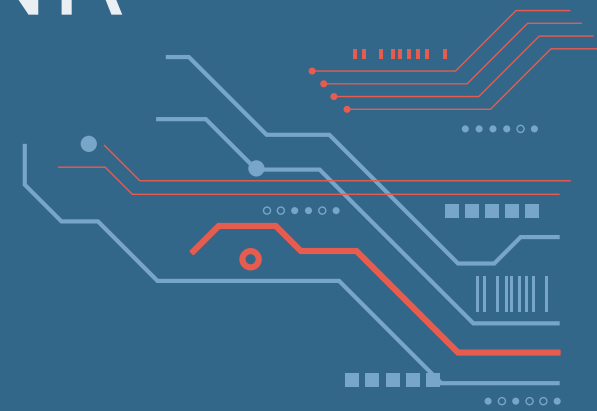


FORMATO DE DATOS TUIA | FCEIA UNR

Docentes | 1C 2026

Esteban Toribio
Iván Pellejero
Juan Pablo Michelino
Renzo Mare
Tomas Ubaldi

toribio@fceia.unr.edu.ar
ipellejero@fceia.unr.edu.ar
jpmich@fceia.unr.edu.ar
rmare@fceia.unr.edu.ar
tomasubaldi@fceia.unr.edu.ar



10



FORMATOS DE DATOS

10.1. ¿Qué es una API?

10.2. Estilos de APIs

10.3. Estilo de arquitectura de APIs

10.4. Comprensión de los formatos de datos

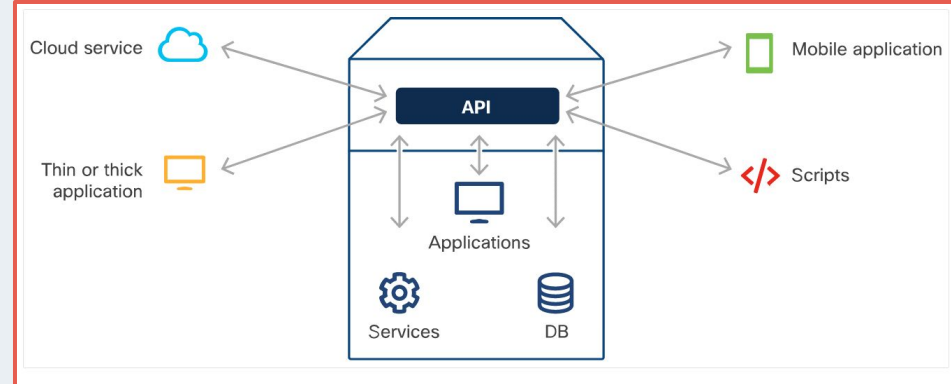


REST APIs

¿Que son?, ¿Por que se usan? y ¿Por que son populares?

¿Que es una API?

- Una interfaz de programación de aplicaciones (API) permite una conversación de software con otra.
- Utiliza interacciones basadas en web o protocolos de comunicación comunes y sus propios estándares patentados.
- Una API determina qué tipo de datos, servicios y funcionalidad expone la aplicación a terceros.
- Al proporcionar API, las aplicaciones pueden controlar lo que exponen de forma segura.



¿Por qué se usan las APIs?

- Las API están diseñadas para ser consumidas mediante programación por otras aplicaciones y también pueden ser usadas por humanos que desean interactuar con la aplicación manualmente.

Los casos de uso de las API son los siguientes:

- **Tareas de automatización:** Crea un script que realice tareas manuales de forma automática y programática.
- **Integración de datos:** Una aplicación puede consumir o reaccionar a los datos proporcionados por otra aplicación.
- **Funcionalidad:** Una aplicación puede integrar la funcionalidad de otra aplicación en su producto.

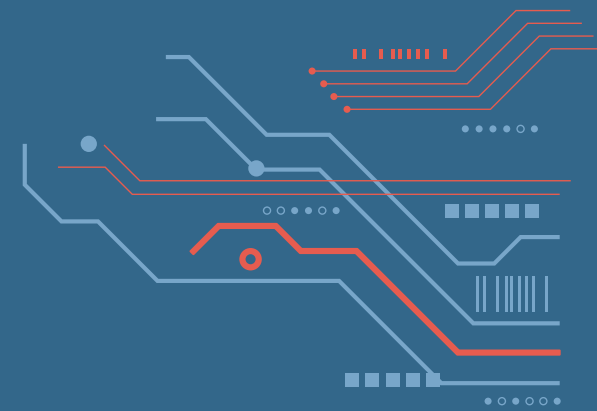
¿Por qué las APIs son populares?

- Las API han existido durante décadas, pero la exposición y consumo de las API han crecido exponencialmente en los últimos 10 años más o menos.
- La mayoría de las API modernas están diseñadas en el producto y se prueban a fondo.
- Los lenguajes de codificación simplificados, como Python, han hecho posible que los ingenieros que no son de software creen aplicaciones y consuman API.

ESTILOS DE APIS

—

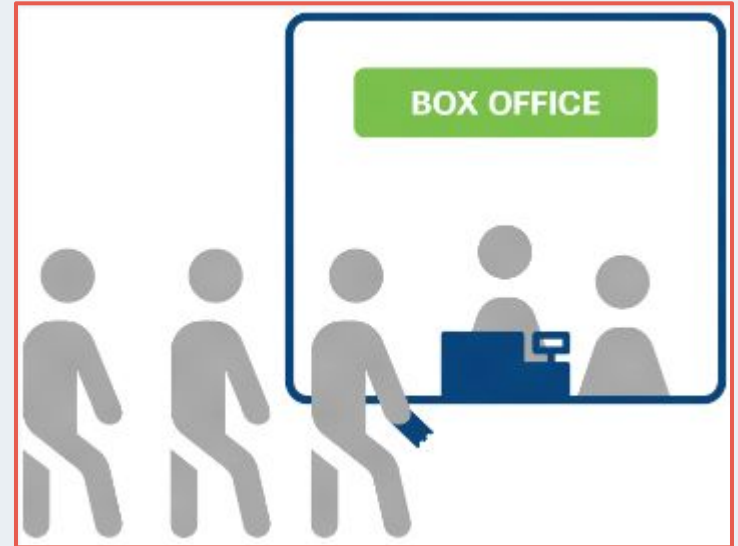
Sincrónicas y Asincrónicas



APIs sincrónicas

Las API sincrónicas responden directamente a una solicitud proporcionando datos inmediatamente.

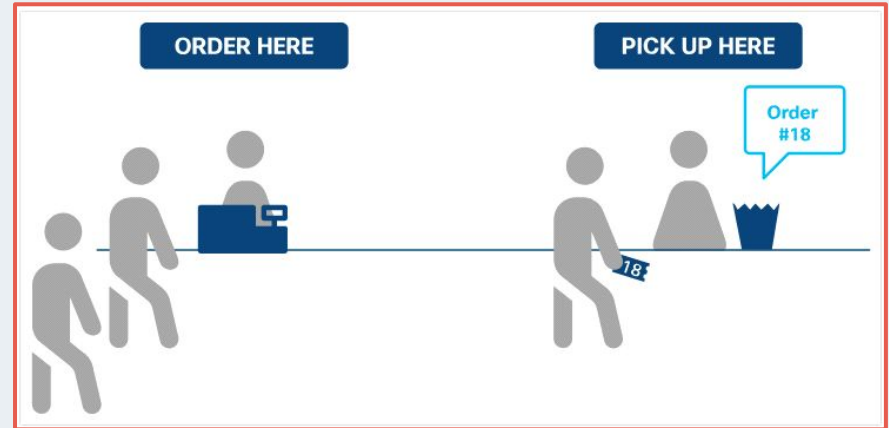
- **¿Cuándo las API son sincrónicas?:** Cuando los datos de la solicitud están disponibles fácilmente.
- **Ventajas de un diseño de API sincrónico:**
 - La aplicación recibe datos inmediatamente.
 - Mejora el rendimiento de la aplicación.
- **Procesamiento del lado del cliente:** La aplicación que realiza la solicitud de API debe esperar la respuesta antes de realizar tareas adicionales de ejecución de código.



APIs asincrónicas

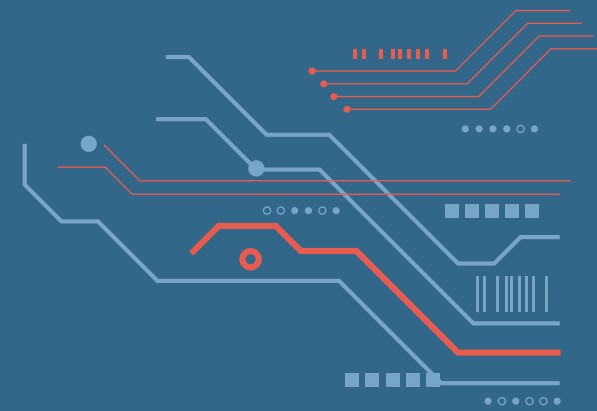
Las API asincrónicas proporcionan una respuesta (sin datos) para indicar que se ha recibido la solicitud.

- **¿Cuándo son asincrónicas las API?:**
Cuando la solicitud tarda algún tiempo en procesarse o si los datos no están disponibles fácilmente.
- **Beneficio del diseño de API asincrónicas:**
La aplicación puede continuar su ejecución sin ser bloqueada hasta que el servidor procese la solicitud.
- **Procesamiento del lado del cliente:** El diseño de la API en el lado del servidor define el requisito en el lado del cliente.



ESTILOS DE ARQUITECTURAS DE APIS

RPC, SOAP, WebSocket y REST



Estilos arquitectónicos

Existen ciertos estándares, protocolos y estilos arquitectónicos específicos que facilitan a los consumidores de la API el aprendizaje y la comprensión de la API.

Los tres tipos más populares de estilos arquitectónicos API son:

- RPC
- SOAP
- WebSocket
- REST

RPC: Remote Procedure Call

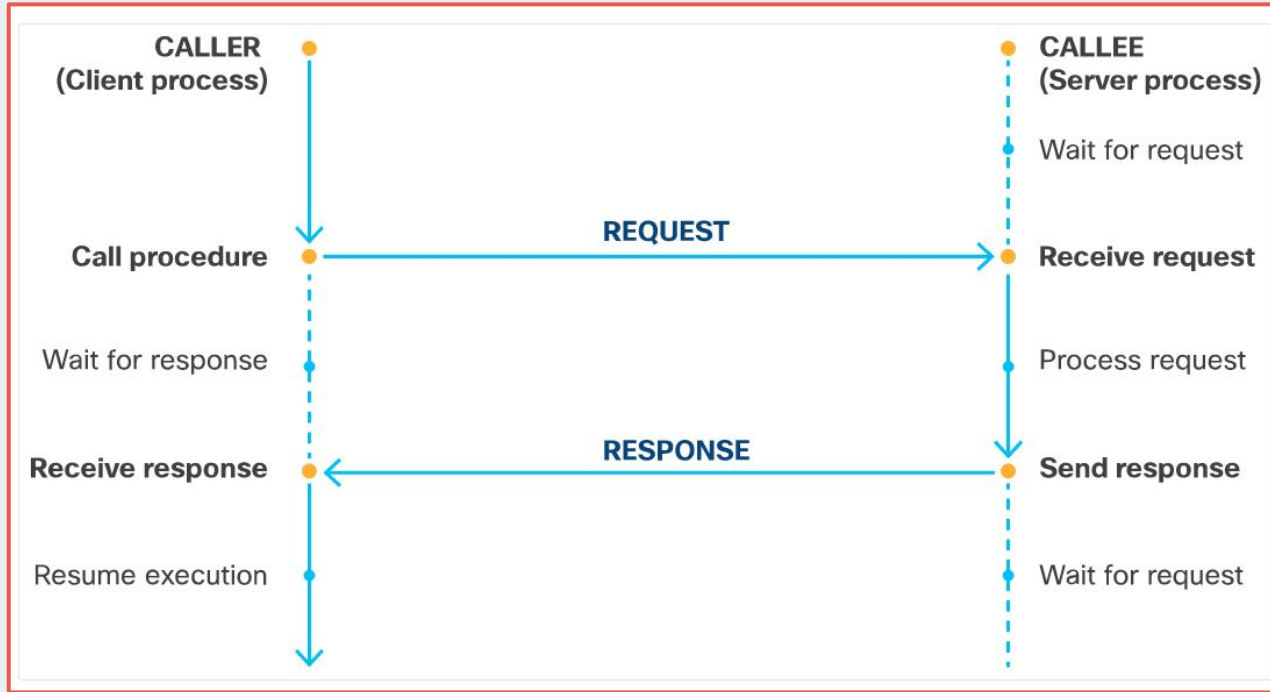
Llamada a procedimiento remoto (RPC) es un modelo de petición-respuesta que permite a una aplicación realizar una llamada a procedimiento a otra aplicación.

Cuando se hace una llamada RPC a un cliente, el método se ejecuta y los resultados se devuelven.

RPC es un estilo API que se puede aplicar a diferentes protocolos de transporte como:

- XML-RPC
- JSON-RPC
- NFS (Sistema de archivos de red)
- Protocolo Simple de Acceso a Objetos (SOAP)

RPC: Remote Procedure Call



Modelo de solicitud/respuesta cliente-servidor de llamada a procedimiento remoto.

SOAP: Simple Object Access Protocol

Protocolo simple de acceso a objetos (SOAP) es un protocolo de mensajería basado en XML. Se utiliza para la comunicación entre aplicaciones en diferentes plataformas o construido con diferentes lenguajes de programación.

SOAP es:

- **Independiente:** Todas las aplicaciones pueden comunicarse entre sí y ejecutarse en diferentes sistemas operativos
- **Extensible:** Añade funciones como fiabilidad y seguridad.
- **Neutral:** puede ser utilizado sobre cualquier protocolo, incluyendo HTTP, SMTP, TCP, UDP o JMS

SOAP: Simple Object Access Protocol

Un mensaje SOAP es un documento XML que puede contener los cuatro elementos siguientes:

- **Sobre:** El elemento raíz del documento XML.
- **Encabezado:** Contiene información específica de la aplicación, como autorización, atributos específicos de SOAP, etc.
- **Cuerpo:** Contiene los datos a ser transportados al destinatario
- **Falla:** Proporciona información de error y/o estado.

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring>Query request too large.</faultstring>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

WebSocket

- La API de WebSocket es otro desarrollo moderno de la API que utiliza objetos JSON para transmitir datos.
- Admite la comunicación bidireccional entre las aplicaciones cliente y el servidor. El servidor puede enviar mensajes de devolución de llamadas a los clientes conectados.
- Para establecer una conexión, el cliente envía una petición de negociación WebSocket y el servidor envía una respuesta de negociación WebSocket.

REST: Representational State Transfer

- Transferencia de Estado Representacional (REST) es un estilo arquitectónico escrito por Roy Thomas Fielding.
- Fielding ha establecido seis restricciones que se pueden aplicar a cualquier protocolo en REST.

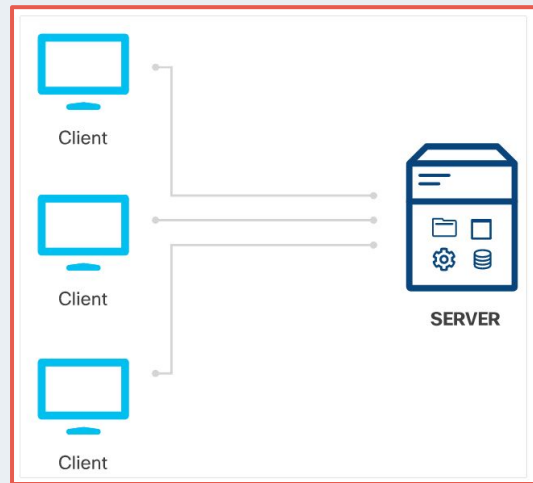
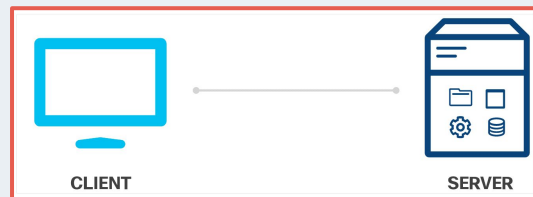
REST: Representational State Transfer

Cliente-Servidor: El cliente y el servidor deben ser independientes entre sí.

Esto permitirá que el cliente se construya para múltiples plataformas, lo que simplificará los componentes del lado del servidor.

Sin estado: El servidor no puede contener estados de sesión.

Las solicitudes del cliente al servidor deben contener el modelo cliente-servidor REST y toda la información que el servidor necesita para realizar la solicitud.



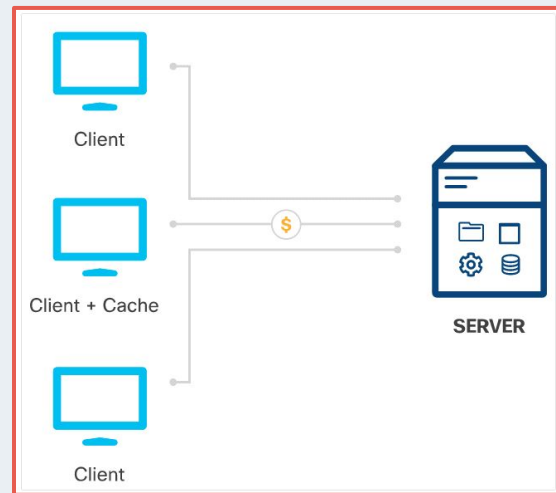
REST: Representational State Transfer

Modelo de caché: Las respuestas del servidor deben indicar si la respuesta se puede almacenar en caché o no en caché.

Si se puede almacenar en caché, el cliente puede utilizar los datos de la respuesta para solicitudes posteriores.

Interfaz uniforme: La interfaz entre el cliente y el servidor se adhiere a los cuatro principios:

- Identificación de recursos
- Manipulación de recursos a través de representaciones
- Mensajes autodescriptivos
- Hypermedia como motor del estado de la aplicación.

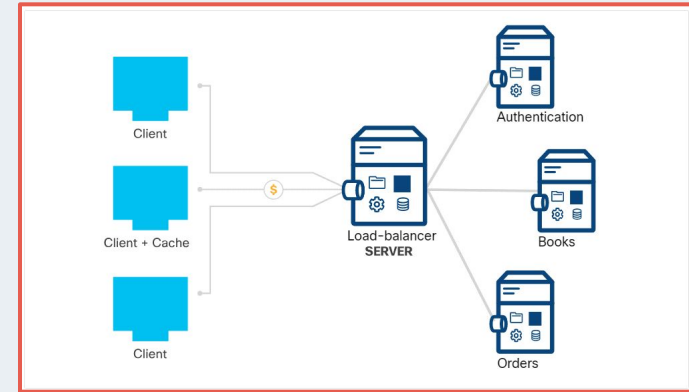


REST: Representational State Transfer

Sistema de capas: Consta de diferentes capas jerárquicas en las que cada una proporciona servicios sólo a la capa que está encima.

Código bajo demanda: La información devuelta por un servicio REST puede incluir código ejecutable (por ejemplo, javascript) o enlaces destinados a extender de manera útil la funcionalidad del cliente.

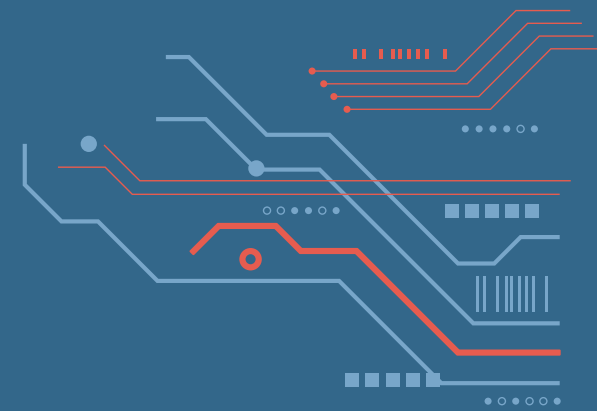
La restricción es opcional porque la ejecución de códigos de terceros introduce posibles riesgos de seguridad.



Comparativas

Características	RPC	SOAP	RESP	WebSocket
Enfoque Principal	Acciones (Invocar funciones/verbos)	Operaciones y Contratos estrictos	Recursos (Sustantivos/URLs)	Canal Bidireccional activo
Protocolo Base	HTTP/1.1, HTTP/2 o TCP	Independiente (HTTP, SMTP, TCP)	Exclusivo sobre HTTP/S	TCP (Inicia con handshake HTTP)
Formato de datos	JSON, XML, Protobuf (binario)	Estrictamente XML	JSON, XML, HTML, Texto plano	Texto plano, JSON, Binario
Estado (State)	Habitualmente Stateless (gRPC permite State)	Stateful o Stateless según diseño	Stateless obligatorio	Stateful (Conexión persistente)
Caso de uso ideal	Microservicios de alto rendimiento	Banca, entornos corporativos legados	APIs públicas de propósito general	Chats, juegos online, dashboards

COMPRENSIÓN DE LOS FORMATOS DE DATOS



XML, JSON y YAML

Formatos de Datos

- Las API Rest le permiten intercambiar información con servicios remotos y equipamiento.
- Los tres formatos estándar más populares para intercambiar información con API remotas son **XML**, **JSON** y **YAML**.
- Un patrón frecuente en las implementaciones de REST API es el siguiente:
 - Autenticar, normalmente mediante la publicación de una combinación de usuario/contraseña y la recuperación de un token que caduca para su uso en la autenticación de solicitudes posteriores.
 - Ejecutar una solicitud GET a un extremo determinado para recuperar el estado de un recurso.
 - Modificar el XML, JSON o YAML devuelto.
 - Ejecutar un POST (o PUT) en el mismo punto final para cambiar el estado del recurso.

XML: Extensible Markup Language

- *EX*tensible *Markup Language* (XML) es una metodología genérica para envolver datos textuales en etiquetas simétricas para indicar semántica.
- Es un derivado de *Structured Generalized Markup Language* (SGML), y también el padre de *HyperText Markup Language* (HTML).
- Los nombres de archivo XML suelen terminar en ".xml"

XML: Extensible Markup Language

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Instance list -->
<vms>
  <vm>
    <vmid>0101af9811012</vmid>
    <type>t1.nano</type>
  </vm>
  <vm>
    <vmid>0102bg8908023</vmid>
    <type>t1.micro</type>
  </vm>
</vms>
```

XML: Extensible Markup Language

- **Cuerpo del documento XML:** Excepto las dos primeras líneas de un documento XML, el resto del documento se considera como el cuerpo.
- **Nombres de etiquetas definidos por el usuario:** Los nombres de etiquetas XML son definidos por el usuario.
- **Codificación especial de caracteres:** Los datos se transmiten en XML como texto legible.
- **Prólogo XML:** El prólogo XML es la primera línea de un archivo XML.
- **Comentarios en XML:** Los archivos XML pueden incluir comentarios, utilizando la misma convención de comentarios utilizada en los documentos HTML.
- **Atributos XML:** XML permite incrustar atributos dentro de etiquetas para transmitir información adicional.

XML: Extensible Markup Language

XML Espacio de Nombres

- Los espacios de nombres son definidos por el IETF y otras autoridades de Internet, organizaciones y entidades, y sus esquemas se alojan normalmente como documentos públicos en la Web.
- Los espacios de nombres se identifican mediante nombres uniformes de recursos (Uniform Resource Names-URI) para hacer que los documentos persistentes sean accesibles sin que el buscador tenga que preocuparse por su ubicación.

```
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">  
  <kill-session>  
    <session-id>4</session-id>  
  </kill-session>  
</rpc>
```

XML: Extensible Markup Language

Interpretación de XML

- En el ejemplo de espacios de nombres XML, la estructura se representa como una lista o array unidimensional (llamada 'instancias') de objetos (cada uno identificado como una 'instancia' mediante etiquetas de corchetes). Cada objeto de instancia contiene dos pares clave-valor que indican un ID de instancia único y un tipo de servidor de Máquina Virtual.
- Una estructura de datos de Python semánticamente equivalente podría declararse como se muestra a continuación

```
vms [  
  {  
    {"vmid": "0101af9811012"},  
    {"type": "t1.nano"}  
  },  
  {  
    {"vmid": "0102bg8908023"},  
    {"type": "t1.micro"}  
  }  
]
```

JSON: JavaScript Object Notation

- *JavaScript Object Notation* (JSON), es un formato de datos derivado de la forma en que se escriben literales de objetos complejos en JavaScript.
- Los nombres de archivo JSON suelen terminar en ".json".
- El ejemplo contiene dos valores que son cadenas de texto, uno es un valor booleano y dos son matrices(Array):

```
{
  "edit-config":
  {
    "default-operation": "merge",
    "test-operation": "set",
    "some-integers": [2,3,5,7,9],
    "a-boolean": true,
    "more-numbers": [2.25E+2,-1.0735],
  }
}
```

JSON: JavaScript Object Notation

- **Tipos de datos básicos JSON:** Incluyen números, cadenas, booleanos o nulos.
- **Objetos JSON:** Al igual que en JavaScript, los objetos individuales comprenden pares clave/valor, que pueden estar rodeados por llaves, individualmente.
- **Mapas y listas JSON:** Cada par clave/valor individual no necesita su propio conjunto de corchetes, pero todo el objeto puede expresar matrices ordenadas por JavaScript (o 'listas') de datos u objetos.
- **Sin comentarios en JSON:** A diferencia de XML y YAML, JSON no admite ningún tipo de método estándar para incluir comentarios en el código.
- **Espacio en blanco insignificante:** El espacio en blanco en JSON no es significativo, y en los archivos se puede emplear sangría utilizando tabs o espacios como se prefiera, o no en absoluto.

YAML: YAML Ain't Markup Language

- *YAML Ain't Markup Language* (YAML) es un superconjunto de JSON diseñado para una legibilidad humana aún más fácil.
- Como superconjunto de JSON, los analizadores YAML generalmente pueden analizar documentos JSON (pero no viceversa).
- YAML es mejor que JSON en ciertas tareas, incluida la capacidad de incrustar JSON directamente (incluidas las comillas) en archivos YAML.

```
---
edit-config:
  a-boolean: true
  default-operation: merge
  more-numbers:
    - 225.0
    - -1.0735
  some-integers:
    - 2
    - 3
    - 5
    - 7
    - 9
  test-operation: set
...
```

YAML: YAML Ain't Markup Language

- **Estructura de archivos YAML:** Se abren convencionalmente con tres guiones (---) y terminan con tres puntos (...).
- **Tipos de datos básicos YAML:** Incluyen números, cadenas, booleanos o nulos.
- **Objetos básicos:** Los tipos de datos básicos se equiparan a claves.
- **Sangría de YAML y estructura de archivos:** YAML indica su jerarquía mediante sangría.
- **Mapas y listas:** YAML representa fácilmente tipos de datos más complejos, como mapas que contienen varios pares clave/valor y listas ordenadas.
 - Los mapas generalmente se expresan en varias líneas, comenzando con una clave de etiqueta y dos puntos, seguidos de miembros, sangrados en las líneas siguientes:

```
mymap:  
  myfirstkey: 5  
  mysecondkey: The quick brown fox
```


YAML: YAML Ain't Markup Language

- **Las listas** (matrices-arrays) se representan con miembros opcionalmente sangrados precedidos por un solo guión y espacio:

```
mylist:  
  - 1  
  - 2  
  - 3
```

- **Los mapas** y las listas también se pueden representar en una llamada "sintaxis de flujo- flow syntax", que se parece mucho a JavaScript o Python:

```
mymap: { myfirstkey: 5, mysecondkey: The quick brown fox}  
mylist: [1, 2, 3]
```

YAML: YAML Ain't Markup Language

- **Cadenas largas:** Se representan usando una sintaxis “plegable”, donde se presume que los saltos de línea son reemplazados por espacios cuando el archivo se analiza o consume, o en una sintaxis no plegada.

```
mylongstring: >
  This is my long string
  which will end up with no linebreaks in it
myotherlongstring: |
  This is my other long string
  which will end up with linebreaks as in the original
```

YAML: YAML Ain't Markup Language

- **Comentarios:** Los comentarios en YAML se pueden insertar en cualquier lugar, excepto en un literal de cadena larga, y están precedidos por el signo hash y un espacio.

```
# this is a comment
```

- **Más características de YAML:** YAML tiene muchas más características, la mayoría de las veces encontradas al usarlo en el contexto de lenguajes específicos, como Python, o al convertir a JSON u otros formatos.

Por ejemplo, para forzar que un número se interprete como una cadena, se puede usar *!!str string*, que es parte del esquema "Failsafe" de YAML

```
mynumericstring: !!str 0.1415
```

Análisis y serialización

- Examinar significa analizar un mensaje, dividiéndolo en sus componentes y comprendiendo sus propósitos en contexto.
- La serialización es más o menos lo opuesto al análisis.
- Los lenguajes de programación populares como Python generalmente incorporan funciones de análisis fáciles de usar que pueden aceptar datos devueltos por una función de E/S y producir una estructura de datos interna semánticamente equivalente que contenga datos escritos válidos.
- En el lado saliente, contienen serializadores que convierten las estructuras de datos internas en mensajes semánticamente equivalentes formateados como cadenas de caracteres.